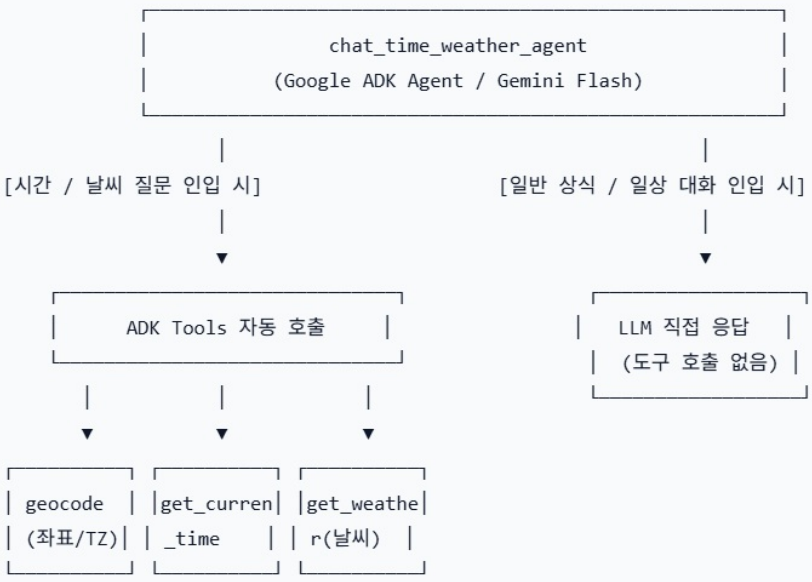
chat_time_weather_agent — 시간·날씨 및 일반 대화 에이전트

`chat_time_weather_agent` 는 Google Agent Development Kit (ADK)를 기반으로 구축된 범용 비서 에이전트입니다. 전 세계 주요 도시의 실시간 현지 시각과 기상 정보를 조회할 수 있는 전용 도구 세트를 갖추고 있으며, 도구 호출이 필요 없는 일반적인 상식 질의, 코딩 질문, 일상 대화도 유연하게 처리합니다.

> 💡 **폴더 명명 규칙 및 ADK 규격 안내**: Python 표준 패키지 명명 규칙(PEP 8) 및 Google ADK의 `AgentLoader` 정규식 규격(`^[a-zA-Z0-9_]+$`)을 준수하여 하이픈(`-`) 대신 언더스코어(`_`)를 사용한 ** `chat_time_weather_agent` **로 구성되어 있습니다. 이를 통해 Python 모듈 `import` 문법 오류와 `adk run` 명령어 로딩 실패를 원천 방지합니다.

---

1. 핵심 기능 및 아키텍처



1. **지능형 지오코딩 (`geocode`)**:
    - Open-Meteo Geocoding API를 호출하여 입력받은 도시 또는 지역명을 위도, 경도, 표준 IANA 시간대(예: `Asia/Seoul` , `America/New_York` , `Europe/Paris`)로 변환합니다.
    - 한국어 주요 도시명(서울, 부산, 제주, 도쿄, 뉴욕, 파리, 런던 등)을 영문으로 자동 변환하는 사전 매핑을 내장하여 검색 정확도를 보장합니다.
    - 네트워크 단절이나 API 오류 시에도 동작할 수 있도록 오프라인 폴백 데이터베이스를 내장하고 있습니다.
2. **정밀 현지 시각 계산 (`get_current_time`)**:
    - 지오코딩으로 확인된 표준 IANA 시간대를 기반으로 Python `zoneinfo.ZoneInfo` 를 적용하여 해당 도시의 실제 현지 시각(UTC 오프셋 및 시간대 약어 포함)을 정확하게 계산합니다.
3. **실시간 기상 정보 조회 (`get_weather`)**:
    - Open-Meteo 실시간 기상 API를 호출하여 WMO 기상 코드(한국어 텍스트로 자동 해석), 기온, 체감 온도, 상대 습도, 풍속을 실시간으로 가져옵니다.
4. **일반 대화 및 상식 질의응답 (General Chat Routing)**:
    - 지시문(Instruction) 프롬프트 엔지니어링을 통해 시간이나 날씨 정보가 불필요한 질의(프로그래밍, 요약, 번역, 일반 상식 등)에는 외부 도구를 전혀 호출하지 않고 모델이 직접 신속하게 응답합니다.

---

2. 파일 구성

```
chat_time_weather_agent/
├── __init__.py           # agent 모듈 및 root_agent 객체 노출
├── agent.py              # geocode, get_current_time, get_weather 도구 및 root_agent 정의
├── README-KR.md          # 한국어 상세 매뉴얼 (본 문서)
└── README-EN.md          # 영문 번역 매뉴얼
```

- `agent.py` :
  - `geocode(location: str) -> dict` : 도시 이름 지오코딩 및 시간대 조회
  - `get_current_time(location: str, latitude: float, longitude: float) -> dict` : 실시간 현지 시각 반환
  - `get_weather(location: str, latitude: float, longitude: float) -> dict` : 현재 기온, 습도, 체감온도, 날씨 상태 반환
  - `root_agent` : Google ADK `Agent` 인스턴스 (모델, 프롬프트, 도구 목록 포함)
- `__init__.py` :
  - `from .agent import root_agent` 형태로 ADK CLI가 인식할 수 있는 엔트리포인트를 제공

---

3. 사전 환경 준비 및 설정

1) Python 환경 및 패키지 설치

Python 3.10 이상 환경에서 ADK 및 필수 라이브러리를 설치합니다:

```
pip install google-adk python-dotenv
# 또는 uv 사용 시
uv pip install google-adk python-dotenv
```

2) 환경 변수(`.env`) 설정

상위 폴더(`part02_architecture` 또는 프로젝트 루트)에 `.env` 파일을 생성하고 Gemini API 키를 설정합니다:

```
# Gemini Developer API 키 사용 시
GEMINI_API_KEY=your_gemini_api_key_here

# 또는 Vertex AI 사용 시
# GOOGLE_GENAI_USE_VERTEXAI=TRUE
# GOOGLE_CLOUD_PROJECT=your-gcp-project-id
# GOOGLE_CLOUD_LOCATION=us-central1

# 선택 사항: 사용할 모델 지정 (기본값: gemini-flash-latest)
ADK_MODEL=gemini-flash-latest

# 선택 사항: 오프라인 테스트 모드 강제 활성화 (0: 온라인, 1: 오프라인 내장 데이터)
AGENT_OFFLINE=0
```

4. 수행 및 실행 방법

방법 1: 터미널 대화형 인터랙티브 모드 (`adk run`)

터미널에서 직접 에이전트와 대화를 주고받을 수 있는 가장 직관적인 실행 방법입니다.

`adk-based_Agent` 폴더에서 실행:

```
# adk-based_Agent 폴더 위치에서
adk run chat_time_weather_agent
```

또는 상위 폴더인 `part02_architecture` 에서 실행:

```
cd ..
adk run adk-based_Agent/chat_time_weather_agent
```

실행 후 프롬프트가 표시되면 질문을 입력합니다:

```
User: 파리 지금 몇 시야?
User: 서울 날씨 알려줘.
User: 파이썬에서 제너레이터가 뭐야?
```

방법 2: 웹 개발 UI 환경 (`adk web`)

Google ADK가 제공하는 공식 브라우저 GUI를 통해 대화, 도구 호출 파라미터, 이벤트 흐름, 세션 상태를 시각적으로 확인합니다.

```
# adk-based_Agent 폴더에서 실행
adk web --port 8008
```

실행 후 웹 브라우저에서 `http://localhost:8008` 로 접속하여 좌측 상단 드롭다운에서
** `chat_time_weather_agent` **를 선택한 후 대화를 진행합니다.

방법 3: 이벤트 생명주기 관찰 스크립트 연계 실행 (`run_events.py`)

ADK의 `Runner` 엔진 내부에서 발생하는 `Event` 객체 흐름(사용자 메시지 ➡️ 모델의 함수 호출 요청 ➡️ 도구 실행 및 결과 반환 ➡️ 모델의 최종 응답)을 코드로 관찰하려면 상위 `Direct_ADK` 폴더의 스크립트를 실행합니다:

```
# part02_architecture 폴더에서 실행
uv run Direct_ADK/run_events.py
# 또는 python Direct_ADK/run_events.py
```

방법 4: 독립형 파이썬 코드로 직접 실행 (Runner 연동)

다음과 같이 간단한 파이썬 스크립트를 작성하여 프로그래밍 방식으로 에이전트를 호출할 수도 있습니다:

```
import asyncio
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai import types
from chat_time_weather_agent.agent import root_agent

async def main():
    # 세션 서비스 및 러너 초기화
    session_service = InMemorySessionService()
    runner = Runner(agent=root_agent, app_name="demo_app", session_service=session_service)

    # 신규 세션 생성
    session = await session_service.create_session(app_name="demo_app", user_id="user_1")

    # 질문 전송
    query = "도쿄 지금 몇 시고 날씨는 어때?"
    msg = types.Content(role="user", parts=[types.Part(text=query)])

    print(f"[사용자 질의]: {query}")
    async for event in runner.run_async(user_id="user_1", session_id=session.id, new_message=msg):
        if event.is_final_response():
            print(f"[에이전트 답변]:\n{event.content.parts[0].text}")

if __name__ == "__main__":
    asyncio.run(main())
```

---

5. 대화 시나리오 및 실행 예시

시나리오 1: 시간 및 날씨 복합 질의 (도구 연쇄 호출)

- **사용자**: "뉴욕 지금 몇 시고 날씨 어때?"
- **에이전트 내부 동작**:
  1. `geocode(location="뉴욕")` 호출 ➡️ 위도 `40.7128`, 경도 `-74.0060`, 시간대 `America/New_York` 획득
  2. `get_current_time(location="New York, United States", ...)` 호출 ➡️ 현지 시각 계산
  3. `get_weather(location="New York, United States", ...)` 호출 ➡️ 기온, 날씨 상태 획득
- **에이전트 최종 응답**:

> 현재 미국 뉴욕의 현지 시각은 2026-11-12 20:00 EST (UTC-0500)입니다. 날씨는 맑음이며, 현재 기온은 12.5°C (체감 온도 11.8°C), 습도는 45%, 풍속은 8.2 km/h입니다.

시나리오 2: 일반 상식 질의 (도구 호출 바이패스)

- **사용자**: "파이썬 데코레이터 원리를 2문장으로 요약해줘."
- **에이전트 내부 동작**:
  - 시간/날씨 질의가 아님을 판단하고 도구를 호출하지 않음 (`function_calls` 없음).
- **에이전트 최종 응답**:

> 데코레이터는 기존 함수의 코드를 직접 수정하지 않고도 부가 기능이나 동작을 덧붙일 수 있게 해주는 파이썬의 문법적 기능입니다. 주로 `@` 심볼을 사용하여 다른 함수를 인자로 받아 새로운 래퍼 함수를 반환하는 고차 함수(Higher-order function) 형태로 동작합니다.

---

6. 문제 해결 (FAQ & Troubleshooting)

현상	원인	해결 방법
`ImportError: cannot import name 'root_agent'`	파일명 또는 폴더명에 하이픈(`-`)이 포함되어 있거나 `__init__.py` 누락	폴더명이 `chat_time_weather_agent` 인지 확인하고 `__init__.py` 가 존재하는지 확인합니다.
`API_KEY_INVALID` 또는 `Gemini key missing`	`.env` 파일에 유효한 API 키가 설정되지 않음	`.env` 에 `GEMINI_API_KEY=...` 를 올바르게 입력했는지 확인합니다.
지오코딩 실패 또는 네트워크 타임아웃	외부 Open-Meteo API 호출 차단 또는 오프라인 환경	`.env` 에 `AGENT_OFFLINE=1` 을 설정하여 내장 오프라인 데이터베이스를 활용합니다.
`adk: command not found`	Google ADK CLI가 가상환경에 설치되지 않음	`pip install google-adk` 실행 후 가상환경이 활성화되어 있는지 확인합니다.